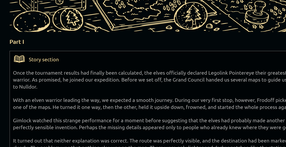
Lord of the Wings [2026]

Everybody Codes is possible thanks to:
Isolution
We specialise in cybersecurity, AI, and IT consulting. We support companies in digital transformation. We are also developing our own product in the field of data security: wizardly.io

Quest 5: The Cartographitty

‹ Quest 4 Quest 6 ›



Part I

Story section

Once the tournament results had finally been calculated, the elves officially declared Legolink *thisterezep* their greatest warrior. As promised, in honor of our expedition, before we set off, the Grand Council handed us several maps to guide us safely to Nulldor.

With an elven warrior leading the way, we expected a smooth journey. During our very first step, however, Frodoff picked up one of the maps. He turned it one way, then the other, held it upside down, frowned, and started the whole process again.

Glimlock watched this strange performance for a moment before suggesting that the elves had probably made another perfectly sensible invention. Perhaps the missing details appeared only to people who already knew where they were going.

It turned out that neither explanation was correct. The route was perfectly visible, and the destination had been marked just as clearly. The problem was that nothing else was on the map. There were only lighter and darker patches, like the static on an old crystal ball that had lost its signal.

Glimlock stared at the map for a while before handing it back. "At least it's consistently elven." Since we would all need to use these maps on the journey to Nulldor, we agreed that everyone should learn how to read them. Legolink offered to explain everything from the beginning.

Legolink explained that the lighter and darker patches were not random at all. To make the maps easier to read, we could represent every lighter cell as `0` and every darker cell as `1`. Every map had an odd number of rows and an odd number of columns. This meant that it always had exactly one central cell.

To reconstruct the land, **begin by drawing a single land cell,** represented by `#`, **in the exact centre of the map.** From that point, new pieces of land are gradually added around the land that has already been discovered. **Consider every cell that touches the current land** horizontally, vertically or diagonally.

A cell becomes land if at least `5` of its eight neighbouring cells are either already marked as land (`#`) or dark cells (`1`) on the original map. Whenever a cell meets this condition, replace it with `#`. It becomes part of the known land and may allow more cells around it to be attached. Continue until no more land cells can be added.

Example based on the following notes:

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

The first step is to mark the central cell of the map as land (`#`). The diagrams below highlight all cells adjacent to the known land, including those touching only at a corner, as these are the only cells that can be added to the land next.

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

After repeatedly attaching every possible cell, the reconstructed land looks like this:

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

After repeatedly attaching every possible cell, the reconstructed land looks like this:

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

The final diagram shows only land cells as `#` and water cells as `.` for better readability. The reconstructed land contains **137** cells.

How many cells on the map are occupied by land?

Your notes for this part: Copy Open Download

Part I solved with answer: **1280**
Check your progress

Part II

Glimlock was still not impressed. He pointed out that such a map could not even tell the difference between solid land and a sandy beach. Legolink calmly picked up another map (**your notes**) and assured us that it could.

First, reconstruct the land exactly as in the previous part. Once the entire land surface has been found, the sandy coastline can be determined.

Every land cell that is **adjacent to at least one water cell,** including diagonal neighbours, immediately becomes sand: `+`. Always treat the map as if it were surrounded by water on all sides, so every cell on the edge of the map is considered adjacent to water.

The remaining sand is then found in much the same way as the land was in the previous part. **A land cell becomes sand** if at least `5` of its eight neighbouring cells are already marked as sand (`+`). Continue until no more cells can become sand.

Example based on the following notes:

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111011000000101011
00100100111011010000
10001101111001010101
00110001110010010000
```

This is exactly the same example as in the previous part, so we already know what the reconstructed land looks like.

```
..#.............
.########..####..
.########..######
..##############.
..##############.
..##############.
..##############.
..##############.
..##############.
.......###.......
```

First, every land cell adjacent to water becomes sand: `+`. This includes diagonal neighbours. The map is also treated as if it were surrounded by water on all sides, so every land cell on its edge becomes sand as well.

```
.+.....+.........
.++++++++..++++..
.++++++++..++++++
..##############.
..##############.
..##############.
..##############.
..##############.
..##############.
.......+++.......
```

The remaining sand is found in much the same way as the land in the previous part. During each step, every land cell with at least `5` neighbouring sand cells also becomes sand.

```
.+.....+.........    .+.....+.........
.++++++++..++++..    .++++++++..++++..
.++++++++..++++++    .++++++++..++++++
..##############.    ..##############.
..##############.    ..##############.
..##############.    ..##############.
..##############.    ..##############.
..##############.    ..##############.
.......+++.......    .......+++.......
```

After the third growth step, no more land cells have at least five sand neighbours, so the process stops. The reconstructed beach contains **69** sand cells.

How many cells on the map are occupied by sand?

Your notes for this part: Copy Open Download

Part II solved with answer: **1383**
Check your progress

Part III

Glimlock stared at the latest map in complete disbelief. "Don't tell me these maps can even show trees. What next, potato stamps?" Legolink quietly opened a small wooden chest filled with... potato stamps. Gimlock blinked. Without a word, Legolink picked up a potato stamp carved into the shape of a tree, then took out another map (**your notes**) to demonstrate the procedure.

First, reconstruct the land and sand exactly as before. Once both have been determined, the remaining land can be decorated with trees. **A tree can be added at `X` wherever the following 5 by 5 pattern fits:**

```
.#+#.
+++++
+#X+#
+++++
.#+#.
```

For every land cell, treat that cell as `X` and examine the area around it:

Dots (`.`) represent cells that are irrelevant. Every `+` marks a cell that must contain either land (`#`) or a tree (`@`). If all cells marked with `+` meet this condition, the land cell at `X` becomes a tree (`@`). Check every land cell in this way.

Once all possible trees have been added with the stamp, **the forest grows for exactly two more steps.** During each step, a land cell becomes a tree if:

- at least `4` of its eight neighbouring cells are already trees, and
- none of its eight neighbouring cells is sand.

Each of these two steps is applied simultaneously across the entire map. This means that trees added during a step may only affect the following step.

Example based on the following notes:

```
01010010100001101000
11110010000011110000
10111101100010101011
01100010101000100000
00011010011001100100
00000111011111010000
10110000101011110000
00111010000001010011
00100100111011010000
10001101111001010101
00110001110010010000
```

This is the same map as in the previous parts, so the land and sand have already been determined. Legolink now checks the remaining land cells from top to bottom and from left to right. Wherever all cells marked with `+` contain land or trees, the cell at `X` becomes a tree (`@`). Trees added earlier may therefore make later stamp positions valid.

The maps below show every successful use of the stamp. The two forest-growth steps are shown separately afterwards.

```
.#.....#.........    .#.....#.........    .#.....#.........    .#.....#.........
.########..####..    .########..####..    .########..####..    .########..####..
.########..######    .########..######    .########..######    .########..######
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
.......###.......    .......###.......    .......###.......    .......###.......
```

```
.#.....#.........    .#.....#.........    .#.....#.........    .#.....#.........
.########..####..    .########..####..    .########..####..    .########..####..
.########..######    .########..######    .########..######    .########..######
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
..##############.    ..##############.    ..##############.    ..##############.
.......###.......    .......###.......    .......###.......    .......###.......
```

During the first growth step, `1` additional land cell becomes tree:

```
.#.....#.........
.########..####..
.########..######
..##############.
..##############.
..##############.
..##############.
..##############.
..##############.
.......###.......
```

During the second growth step, `2` additional land cells become trees:

```
.#.....#.........
.########..####..
.########..######
..##############.
..##############.
..##############.
..##############.
..##############.
..##############.
.......###.......
```

The final map contains **14** tree cells.

How many cells on the map are occupied by trees?

Your notes for this part: Copy Open Download

Part III solved with answer: **9865**
Check your progress

Puzzle solved! Don't stop now!

Post your solution, compare ideas, and help others grow on Reddit or join us for a chat on Discord

Bonus for Supporters!

This feature is available only to Supporters. Be a small "thank you" for supporting this event. Click the link below to **get all 100 versions of the input notes prepared for this Quest,** along with the answers! Please don't host it anywhere; especially please don't add it to your solutions repo. Thank you for being awesome!

Download

© 2024-2026 Everybody Codes. All rights reserved.
Terms of Use, Privacy Policy, Cookies and Tracking Policy
By using this website, you agree to these terms.